Adaptive Lightweight Network Intrusion Detection System (NIDS)

Comprehensive Project Defense, Architecture & Technical Presentation Guide

1. Executive Summary & Problem Motivation

Modern edge gateways, IoT perimeters, and cloud firewalls process thousands of network packets per second. Traditional Network Intrusion Detection Systems (NIDS) suffer from a critical performance-accuracy dilemma: Deep Learning architectures (LSTMs, CNNs, Transformers) offer high representation capacity but require excessive memory and introduce 10–50+ ms inference latencies per packet, rendering them unfeasible for real-time edge hardware. Conversely, single standalone ML models struggle with noise and fail to generalize against novel attack vectors.

The Adaptive Lightweight Solution: This project bridges this critical gap by delivering high detection accuracy with sub-1.5ms inference latency through a unified system combining intelligent dimension reduction, heterogeneous tree ensembles, adaptive dynamic weighting based on information entropy, zero-day anomaly scoring, and production cloud microservices.

> ***Core Innovation:*** *Dimension reduction from 49 to 15 features reduces latency by ~4x (4.85ms → 1.25ms) and memory by 64%, while our Adaptive Dynamic Weighting outperforms static soft voting by +1.7% in F1-score.*

2. End-to-End System Architecture (Stages A to F)

- **Stage A: Data Ingestion & Extraction:** Ingests raw packet flows or standard benchmark datasets (UNSW-NB15 / CICIDS2017), capturing 49 high-dimensional flow statistics (flow duration, inter-arrival times, packet rates, TCP state flags, byte volume, TTL, jitter).
- **Stage B: Data Preprocessing Pipeline:** Sanitizes flow records via clean.py by imputing missing values and replacing infinities (+/-inf) with training medians. encoder.py transforms categorical attributes (proto, service, state) with fallback for unseen tokens, followed by StandardScaler normalization.
- **Stage C: Intelligent Dimension Reduction (Top 15 Features):** Applies a hybrid scoring algorithm combining Mutual Information (non-linear dependency) and Random Forest Gini Feature Importance (tree split gain). Prunes the feature space from 49 to 15 features, eliminating redundancy and reducing memory footprint.
- **Stage D: Heterogeneous Tree Ensemble:** Passes the 15-feature vector through three complementary base classifiers in parallel: Random Forest (bagging variance reducer), XGBoost (gradient boosting boundary discriminator), and CatBoost (categorical variance optimizer).
- **Stage E: Adaptive Dynamic Weighting:** Instead of static 33% weights, dynamically calculates confidence margins (P_top1 - P_top2) and Shannon Entropy (uncertainty) per packet. Dynamically assigns higher voting influence to the model displaying maximum certainty on that specific flow.
- **Stage F: Zero-Day Anomaly Detection & Telemetry:** Monitors prediction distribution entropy. Flows exhibiting high entropy (H > 1.1) or low confidence (< 0.60) are intercepted and categorized as UNKNOWN / ANOMALOUS. Results are broadcast via WebSockets to a React SOC dashboard in real time.

3. Mathematical & Algorithmic Formulations

1. Hybrid Feature Selection Score:

Score(f) = 0.5 × [(MI(X_f; Y) – min(MI)) / (max(MI) – min(MI))] + 0.5 × [(Gini(f) – min(Gini)) / (max(Gini) – min(Gini))]
Balances information-theoretic dependency with empirical decision tree split significance.

2. Sample-Wise Dynamic Ensemble Weighting:

Margin_m = P_m(class_1) – P_m(class_2) | Entropy H_m = – ∑ p_i ln(p_i)
Confidence S_m = BaseWeight_m × (Margin_m + 0.1) / (1.0 + H_m) | Weight_m = S_m / ∑ S_j
Ensures confident classifiers dynamically overpower uncertain models on difficult boundary flows.

4. Experimental Benchmarks & Ablation Study

Automated ablation benchmarks comparing the complete Adaptive Lightweight Ensemble against individual models and the unreduced 49-feature baseline:

Model Configuration	Features	Accuracy	F1-Score	Latency (ms)	RAM Footprint
Adaptive Ensemble (Ours)	15	**98.4%**	**0.982**	**1.25 ms**	**~245 MB**
Static Voting Ensemble	15	96.8%	0.965	1.28 ms	~245 MB
Random Forest Only	15	95.9%	0.957	0.95 ms	~180 MB
XGBoost Only	15	96.1%	0.959	0.88 ms	~140 MB
CatBoost Only	15	96.4%	0.962	1.10 ms	~160 MB
Full Model (No Selection)	49	98.6%	0.983	4.85 ms	~680 MB

5. Live Cloud Deployment & Demonstration Links

The system has been containerized and deployed into production cloud infrastructure:

- **Frontend SOC Dashboard (Vercel Global Edge):** https://nids-project-neon.vercel.app
- **Backend API & Interactive Swagger (Render Cloud):** https://nids-project-bbsy.onrender.com/docs
- **Public GitHub Repository:** https://github.com/satwikpkp/nids-project
- **Default Operator Credentials:** admin@soc.defense / Admin@123456

6. Key Defending Points for Oral Examination

Q: Why tree-based ensembles instead of Deep Learning (LSTM/CNN)?
Tabular flow statistics do not exhibit continuous spatial/temporal dependencies like images or text. GBDT (XGBoost/CatBoost) and Random Forest consistently achieve higher AUROC on tabular network data with >10x faster inference speed and sub-250MB RAM footprint.

Q: How does the system handle zero-day attacks?
Through our out-of-distribution entropy scoring hook. Novel attacks exhibit high prediction entropy and low margin

confidence across our heterogeneous models, triggering an UNKNOWN / ANOMALOUS alert.

Q: What is the computational benefit of feature reduction?
Memory footprint is reduced by 64% and latency decreases from 4.85 ms to 1.25 ms per packet (a nearly 4x speedup) with less than 0.2% change in F1-score.